

EEET2588/EEET2687

Real-Time Systems Engineering

Lecture 1: Introduction to Real-Time Systems

Dr Linh Tran - School of Science, Engineering & Technology



Dr Linh Duc TRAN

Course Coordinator & Main Lecturer

- Ph.D., M.Eng, B.Eng.(Hons), GradCertBA
- Lecturer of Software Engineering & Information Technology Programs at RMIT Vietnam
- Research Interests: Programming, Embedded and IoT Applications, Computer Architecture and Protocols, GenAI in Education.
- Industry experience:
 - Two years in the industry as Embedded Firmware Engineer – AppliedMicro Corporation, 2013 – 2015 (now is MACOM).
 - STEM Education Practitioner – Advisor and Trainer, conduct training on integrating Science and Engineering practice in curriculum (2018 – 2019)
 - Researcher and Project Lead in multiple R&D projects.
- Email: linh.tranduc@rmit.edu.vn
- Consultation Time: email for appointment.





Dr Hung Viet PHAM

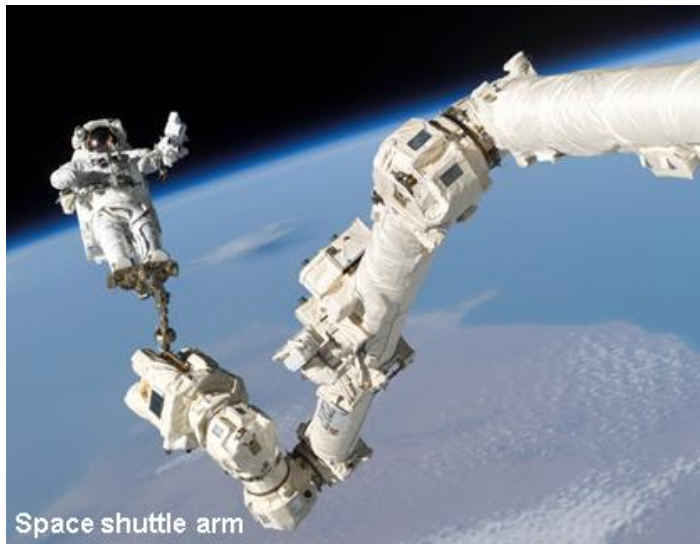
Lecturer (Hanoi Campus)



- Ph.D., M.Eng, B.Eng.(Hons), GradCertBA
- Associate Program Manager, Electronic Computer Systems & Robotics Mechatronics Engineering
- Dr Hung received his Master's degree from RMIT Vietnam in 2011. He then worked as an engineer at Intel Vietnam, focusing on test and assembly processes, and contributed to bringing the first CPU production line to Intel Vietnam. He later completed his PhD at RMIT University, producing significant research outcomes.
- Industry experience:
 - Worked with cross-functional departments to deliver key outcomes.
 - Developed failure criteria, technical specifications, and training materials, and delivered training to technicians.
 - Successfully transferred the first CPU product and multiple strip-processing products from Malaysia to Vietnam.
- Email: hung.phamviet2@rmit.edu.vn
- Consultation Time: email for appointment.

OUTLINE OF TODAY'S LECTURE

1. **Course Overview.** Syllabus outline, Assessment Schedule
2. **What is a real-time system?** Deadlines, hard / soft / firm, concurrency, scheduling
3. **Real-time operating systems.** What an RTOS gives you, POSIX, synchronization, IPC, design
4. **QNX Neutrino.** The microkernel RTOS you will use, Momentics, your first program



Course Description

EEET2588/EEET2687 Real-Time Systems Engineering is designed to build your foundation in real-time embedded systems and software development, where systems must produce the correct result at the correct time. These systems are commonly used in cars, aircraft, medical devices, and industrial control.

The course covers concurrency and how a real-time operating system manages it, including processes and threads, priority-based pre-emptive scheduling, synchronization, inter-process communication, common hazards such as deadlock and priority inversion, and system design using state machines and UML.

The course is hands-on and built around the QNX Neutrino RTOS. It combines lectures with six laboratories and a practical group project to help you consolidate your understanding and apply it to real-life scenarios.

WHY ARE WE HERE...

Embedded engineer in Vietnam

391 results

Set alert ☐



Your profile matches this job

Promoted • 18 applicants



Layout Design Engineer, A&MS - 41663BR

Synopsys Inc

Đà Nẵng, Đà Nẵng City, Vietnam



3 connections work here

Promoted



Hardware PCB SI Engineer, Sr Staff

Marvell Technology

Hồ Chí Minh City, Hồ Chí Minh City, Vietnam



Actively recruiting

2 weeks ago



Embedded Software Engineer (Medical Domain)

HCLTech Vietnam

Hanoi, Hanoi, Vietnam (On-site)



Actively recruiting

Promoted • 2 applicants • [Easy Apply](#)



Diagnostic Firmware Engineer

Ampere

District 7, Hồ Chí Minh City, Vietnam (On-site)



Actively recruiting

Embedded Software Engineer (Medical ...

HCLTech Vietnam · Hanoi, Hanoi, Vietnam (On-site)

[Easy Apply](#)

[Save](#)



Location: Hanoi Office - 643 Pham Van Dong, Tu Liem North.

DUTY & RESPONSIBILITIES

- Technical ownership of sub-system / module(s) development.
- Understands system / sub-system requirements and elicit requirements for module(s) and maintains traceability of requirements in the life cycle.
- Understands solution design of system / sub-system development, interfaces, integration and verification strategy.
- Hands-on in development – develops efficient and zero-defect code, complying to coding standards, and with usage of required tools.
- Capable of trouble shooting technical issues and resolve issues immediately.
- Technical peer review expertise and works with team members towards optimal and efficient solution.
- Communicates technical issues / challenges and works towards resolution.
- Has exposure to latest trends / technologies / tools and upgrades skills proactively.
- Exposure working with distributed teams to meet the common objective.

REQUIREMENTS

Must have

- Extensive experience in **Embedded C & C++ programming (4+ year)**.
- Experience with **UML modelling expertise** (mandatory).
- Extensive experience working on **microcontrollers** (32 bit / 64 bit) – well versed in usage of IAR workbench or Microcontroller Development Kit (MDK).
- Extensive experience on **RTOS** (Linux, QNX, RTX, FreeRTOS or other RTOS) – two or more RTOS experience.
- Extensive experience on embedded software development life cycle.
- Requirement, Design, Development, Verification (Unit level, Integration and Software System Level) & Validation.
- Usage of static code analyzer tools (QAC / QAC++) and other tools (testing,

COURSE SCHEDULE

Week	Module	Activities and Assessments
1	Introduction to Real-Time Systems	Getting Started with QNX
2	Concurrent Processes	Lab 1
3	Concurrent Processes (cont'd)	Lab 2 (Lab 1 report due)
4	Data Protection Theory: Mutual exclusion operations	Lab 3 (Lab 2 report due) <i>In-class Quiz 1 (early feedback)</i>
5	Data Protection and Synchronization	Lab 4 (Lab 2 report due)
6	Real-Time System Structured Design and UML Diagrams	Lab 5 (Lab 3 report due) <i>In-class Quiz 2</i>
7	<i>Personal Development Week (non-teaching)</i>	
8	Task Structuring Criteria	Lab 6 (Lab 5 report due)
9	Task Structuring Criteria (cont'd)	Lab 6 report due <i>Assessment 2: Group Project (Initial Design Report)</i>
10	Resource Concepts and Deadlock	<i>Assessment 3: In-class Written Test (25%)</i>
11	Resource Concepts and Deadlock (cont'd)	
12	Real-Time Scheduling	<i>Assessment 2: Group Project (Project Implementation)</i>

ASSESSMENT SCHEME

Assessment items	Weight
Laboratory Assessment (Early Feedback, Individual)	15%
In-class Quizzes (Individual)	10%
Group Project: - Initial design - Final Report & Implementation	Total 50% • 10% • 40%
Written Assessment (Individual)	25%

- The group project include some individual component (contribution evaluation, individual performance during demonstration, etc.)
- Late penalties: late work will be penalised at 10% per day after the deadline. After 3 days late, a zero score is recorded.

Note: *All assessment task need to comply with RMIT's Student Conduct policy*

ACADEMIC MISCONDUCT

- **Plagiarism:** the presentation of the work, idea or creation of another person as though it is your own. Plagiarised material can be drawn from and presented in written, graphic, visual, electronic or oral form. Plagiarism occurs when the origin of the materials used is not appropriately cited.
- **Enabling Plagiarism:** the act of assisting or allowing another person to plagiarise or to copy your own work.
- **Cheating in Assessments:** attempting to gain an unfair advantage in an invigilated assessment, breaching assessment rules, or violating assessment instructions in a way that defeats or compromises the purpose of the task.
- **Contract Cheating:** submitting work completed by another person or from an online study platform as your own, or completing an assessment task for another person.
- ***All academic misconduct may lead to failure for the assignment, failure for the course, suspension, or exclusion from the university.***

Don't forget: The lecturers are there to answer your questions! If you are not sure what is allowed or prohibited, ask the lecturer!

HOW TO SUCCEED IN THIS COURSE

- Read, and try to understand all terms before coming to lecture.
- Attend all class sessions (on time!). Pay attention and participate in class activities via talk, chat, reactions, emojis.
- The amount of info can be overwhelming at the first time you read it. Please take note from the lecture, you can know the key takeaway for each slide/module.
- If you still feel dizzy? Do the tutorial, it will clear your mind.
- Spend time on exploring and reading extra materials on the topic. There will be many new concepts that will take time to digest.
- Review lecture, your own notes and tutorial exercises (continue to do it at home if not yet completed in class).
- Form and work with your group as soon as possible for the group assignment.
- No plagiarism or cheating in any assessment !
- ASK QUESTIONS !

CANVAS AND COMMUNICATION WITH STAFF

- Check your student email and Canvas regularly. **Don't miss any announcement** (*make sure you turn on Email Notification for them in Course Notification Settings*).
- For any issues regarding to the course, you are encouraged to contact the lecturer. Don't wait until TOO LATE to discuss your problems and difficulties.
- **Class Discussion, Q and A** channel: to ask/ discuss about learning materials, lab exercises, team formation, etc.
- Please use **EMAIL** for personal difficulties/ issues instead of Canvas Inbox since the Canvas message may not create a notification and sometime disappear into a black hole! Please don't make the emails too long if not necessary. Be concise.

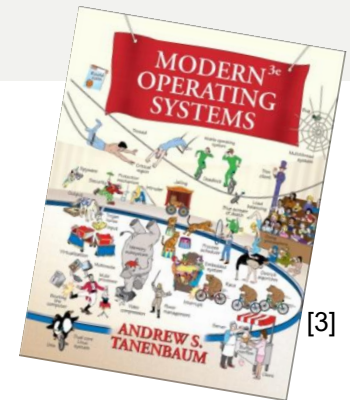
QNX 7.1

- We will use QNX 7.1 in this course. Create a QNX account by selecting "New member" at: <https://www.qnx.com/account/login.html>
- After your account has been verified, please add your email address to the spreadsheet (link provided on Canvas).
- Once the email address on the google spreadsheet has been processed you will be able to download and install your own copy of **QNX Software Center** and then install the **QNX Software Development Platform 7.1** and **QNX Momentics IDE** package on your own personal computer.
- Detailed instruction of installation and getting started is provided at the page "[Getting started with QNX at RMIT](#)" on Canvas.

Reference Materials

- **Main references:**

- Link to download QNX 7.1 Documents:
<https://www.qnx.com/download/group.html?programid=51217>
- Concurrent Processes & Operating Systems
 - *Modern Operating Systems*, Tanenbaum, Andrew S., Prentice-Hall
 - *Getting Started with QNX Neutrino* (most up-to-date version on Canvas)
 - *QNX Neutrino Programmer's Guide* (most up-to-date version on Canvas)



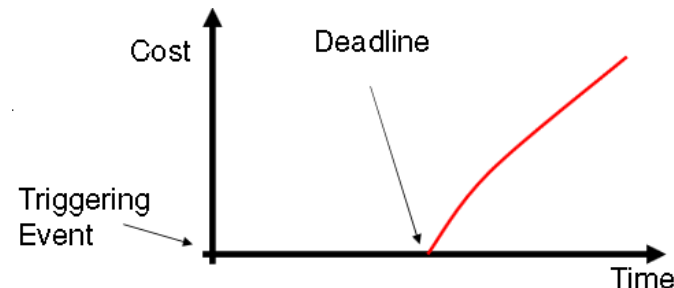
- **Other useful texts:**

- Beej's Guide to C Programming", ebook freely available on Canvas shell
- Teach Yourself C++ in 21 Days", ebook freely available on Canvas shell
- Real-Time Systems Design, Scheduling
 - *Designing Concurrent, Distributed and Real-Time Applications with UML*, Gomaa, Hassan, Addison-Wesley
 - *Real-Time Systems Development*, Williams, Rob, Elsevier
- Reliability and Fault Tolerance
 - *Real-Time Systems and Programming Languages*, Burns, Alan & Wellings, Andy, Addison W.

What is a Real-Time System?

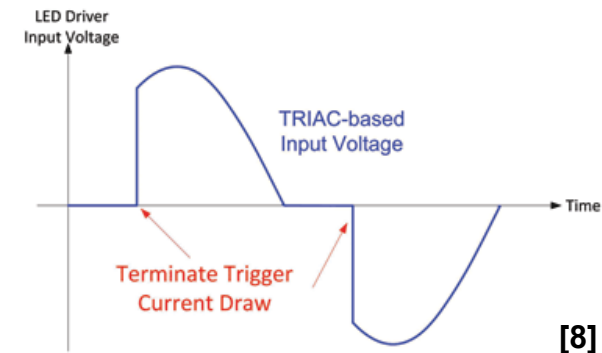
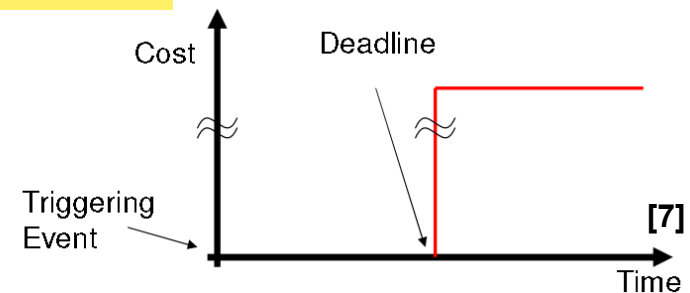
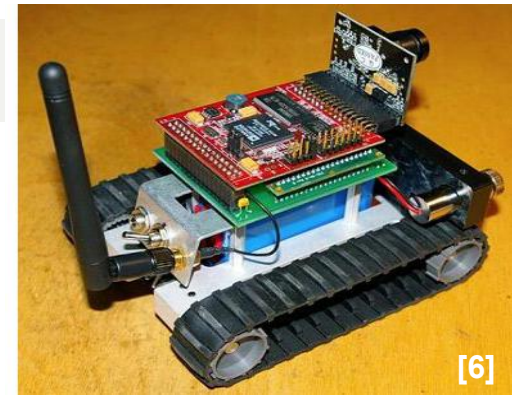
Real-Time Systems

- The **correctness of the result** depends not only on the **results of computation** but also on the **time** at which those results are produced
- Must be, Reliable and Predictable
 - A deadline is given time after a trigger event, by which a response has to be completed
- **Hard Real-Time Systems**
 - Failure to respond can be catastrophic
 - Must meet deadlines
 - **Flight control system, Nuclear Power plant control**
- **Soft Real-Time Systems**
 - Failure to respond is annoying
 - May miss some deadlines
 - **Web site service, display board, telecom**
- **Firm Real-Time Systems**
 - If failure occurs, system does not fail
 - Results produced after deadline are rejected (loss of revenue).
 - **Video conferencing (QoS)**



Real-Time Computing

- Real-Time
 - The computer must respond to stimuli in a timely fashion
- **Stimuli**
 - Derived from analog signals, digital signals, peripheral devices
- **Timely**
 - Not necessarily quickly but relative to time-scale of process being controlled
 - Before user complains to (or abuses) the operator
 - Before boiler blows up, or something bad happens
 - Before chemical process is spoilt
- Real-time computing is not the answer to everything
- **Not suitable for applications** such as:
 - High voltage phase control applications (i.e. near instantaneous response)
 - CPR systems (heart beat detection can be done easier by dedicated electronics)
 - Simple state machines that don't need multiple subsystems (avoid complexity & overkill)



Real-Time Systems Domains

- **Data processing**

- Financial (banks, stock brokers, etc)
- Reservations (airline, theatre, etc)
- ASX, TAB, Myki, etc. (data networks)
- GPS navigation

- **Industrial Process Control**

- Chemical processing
- Oil Refineries
- Power Generation
- Production Line

- **Signal Processing**

- Audio
- Images

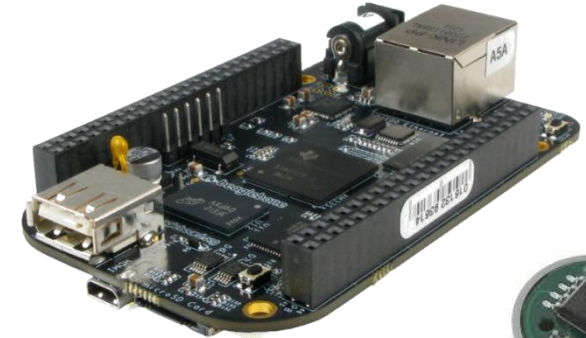
- **Safety-Critical Systems**

- Aerospace and Military
- Vehicle control

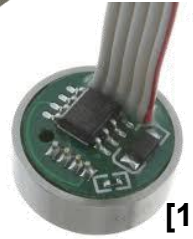


Real-Time Systems Hardware

- Microprocessors (PCs)
- Sensors (pressure, temperature etc)
 - Require analog/digital, parallel interfaces
- Controllers (of motors, valves etc)
 - Require digital/analog, parallel interfaces
- Communications
 - Serial, parallel, bus, networked
- Operator and Monitor Consoles
- Data Logging (printers, disk, tape etc)



[12]



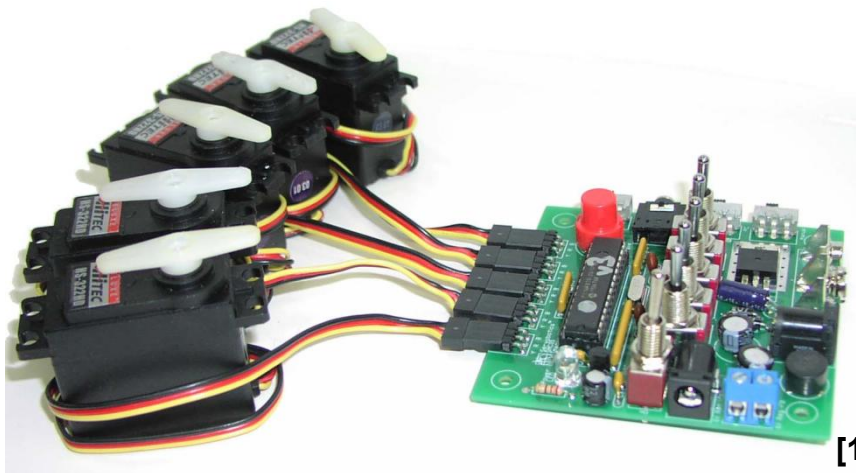
[14]



[13]



[15]



[18]



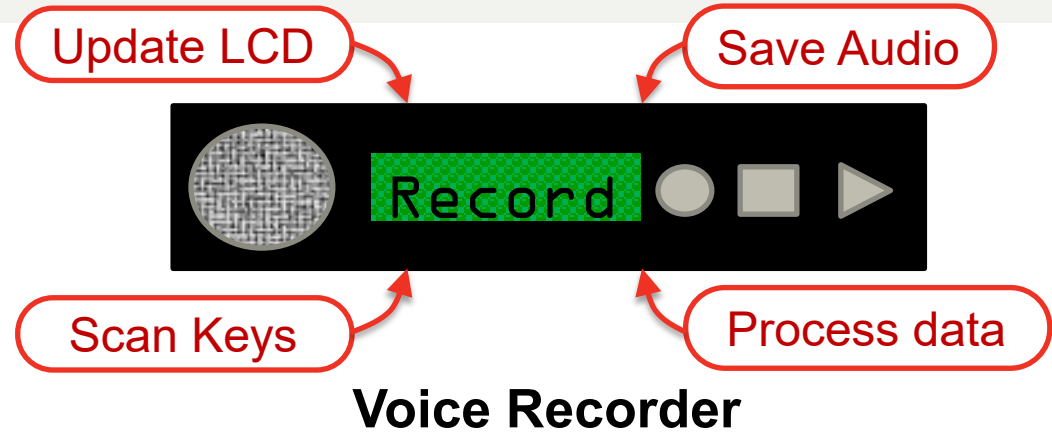
[17]



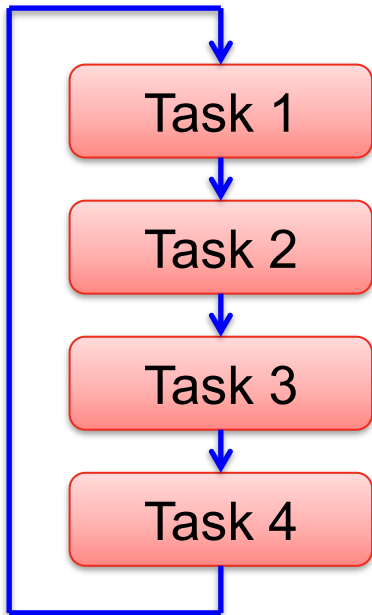
[16]

Software Concurrency

- Sequential tasks are good for a simple system, however what happens if we need to update the LCD screen while recording audio and scanning for a key press?

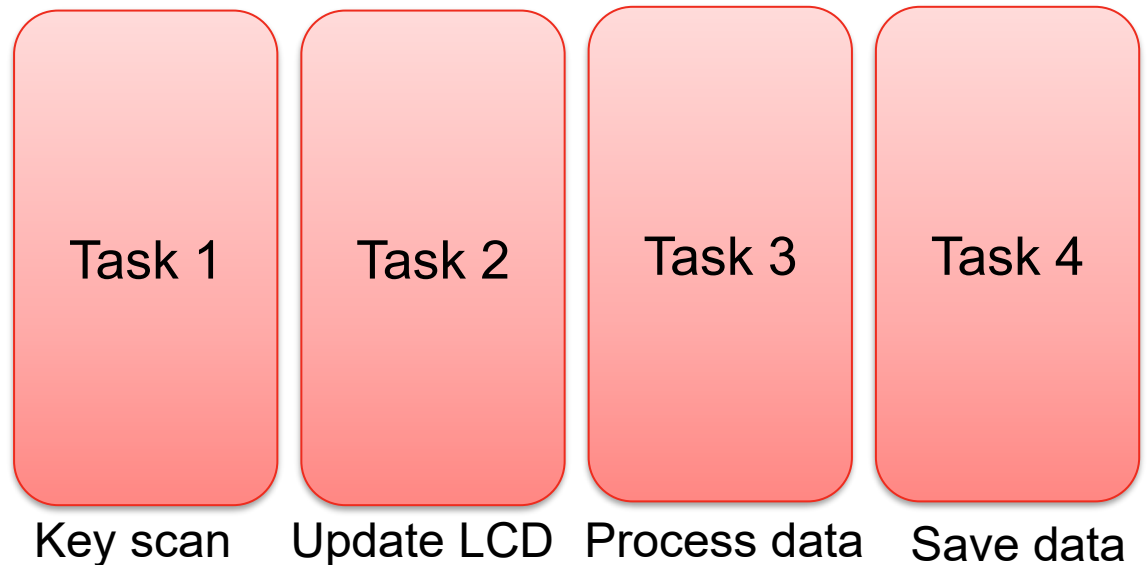


Sequential Tasks



Tasks appear to be running concurrently

(although each task may be divided into small chunks and the CPU switches between chunks from different tasks)

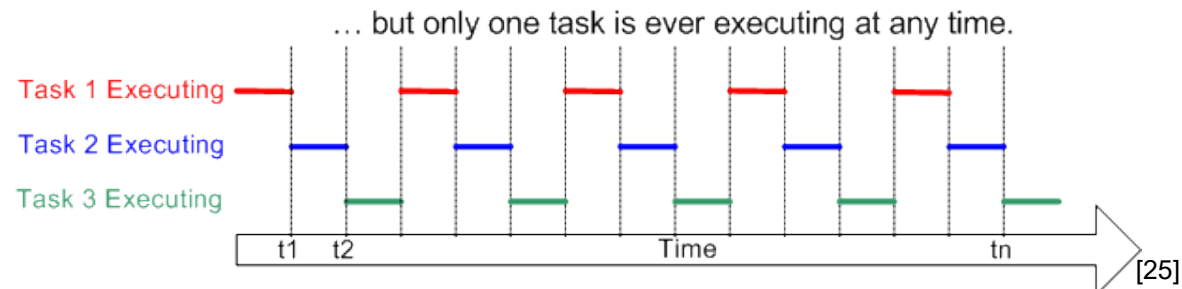
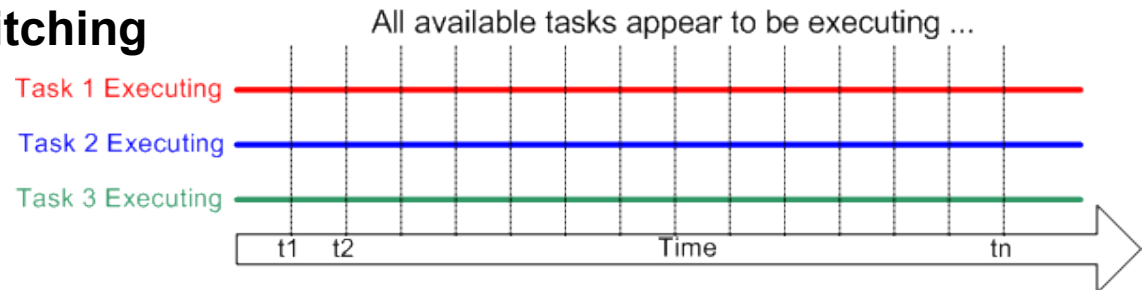


Protection, Privileges and Context Switching

- **Main job of an OS is to provide protection from errant user programs**
 - User programs should be prevented from accessing memory and I/O devices outside their allocated bounds
 - Typically provide **two or more levels of protection**
 - Prevent non-privileged user processes from accessing anything that would affect the integrity of the operating system
 - Prevent process from accessing other user processes.
 - This includes preventing direct access to I/O devices, and disallowing the use of certain instructions e.g. Halt (the CPU), Enable or Disable Interrupts.
 - Privilege level changed on system calls and hardware interrupts

- **OS provides fast context switching**

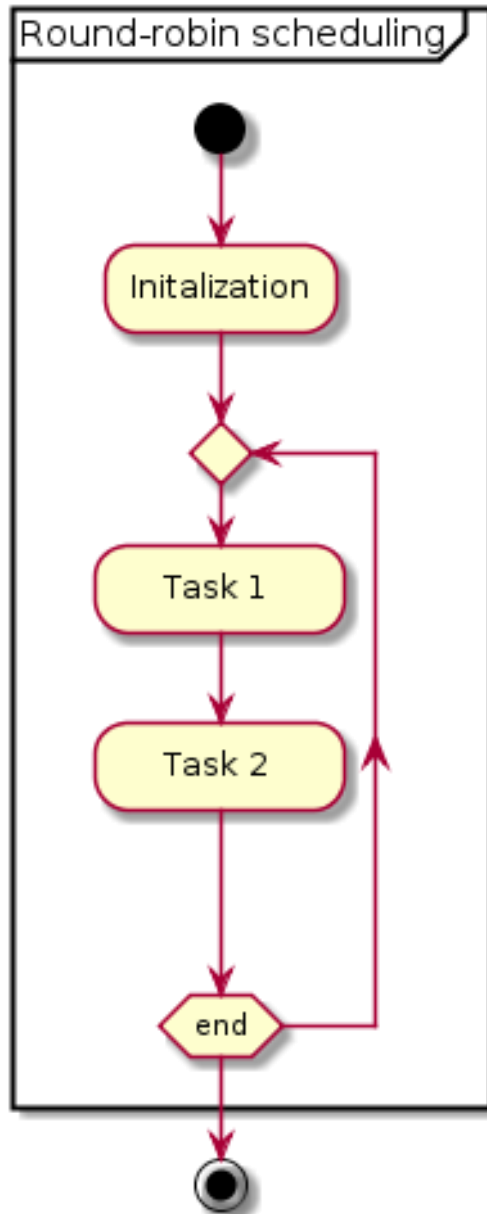
- Tasks appear to execute concurrently
- RTOS provided means of communication between threads (if required)
- RTOS scheduling should be user selectable.
ie: FIFO, RR, Sporadic



Processes vs Threads

- **Process** = an independent program in execution, with its own protected memory space and process ID (pid). Processes share data explicitly: message queues, channels, named semaphores.
- **Thread** = a unit of execution inside a process, each with its own thread ID (tid). Threads in one process share the same memory, so they exchange data through shared variables – fast, but unsafe without protection.
- **QNX / POSIX building blocks (used from Lab 2 onward):**
 - `pthread_create()`, `pthread_join()` – spawn a thread and wait for it
 - Priority 1–63 (default 10); scheduling policies `SCHED_RR` (round-robin) and `SCHED_FIFO`
 - `getpid()`, `gettid()`; inspect live thread state on the target with the `pidin` command
- **Rule of thumb:** threads for tightly-coupled work sharing data on one node; processes for isolation, fault containment, or work spread across QNX nodes.
- *Read more: QNX System Architecture – The Kernel:
http://www.qnx.com/developers/docs/7.1/#com.qnx.doc.neutrino.sys_arch/topic/kernel.html*

Bare-metal Scheduling



```
int main(int argc, char *argv[])
{
    System_Init(); // takes 250 ms

    // Begin round robin scheduling
    while(!end)
    {
        // Task 1 must run every 5 seconds (at least)
        Task1(); // takes 500 ms

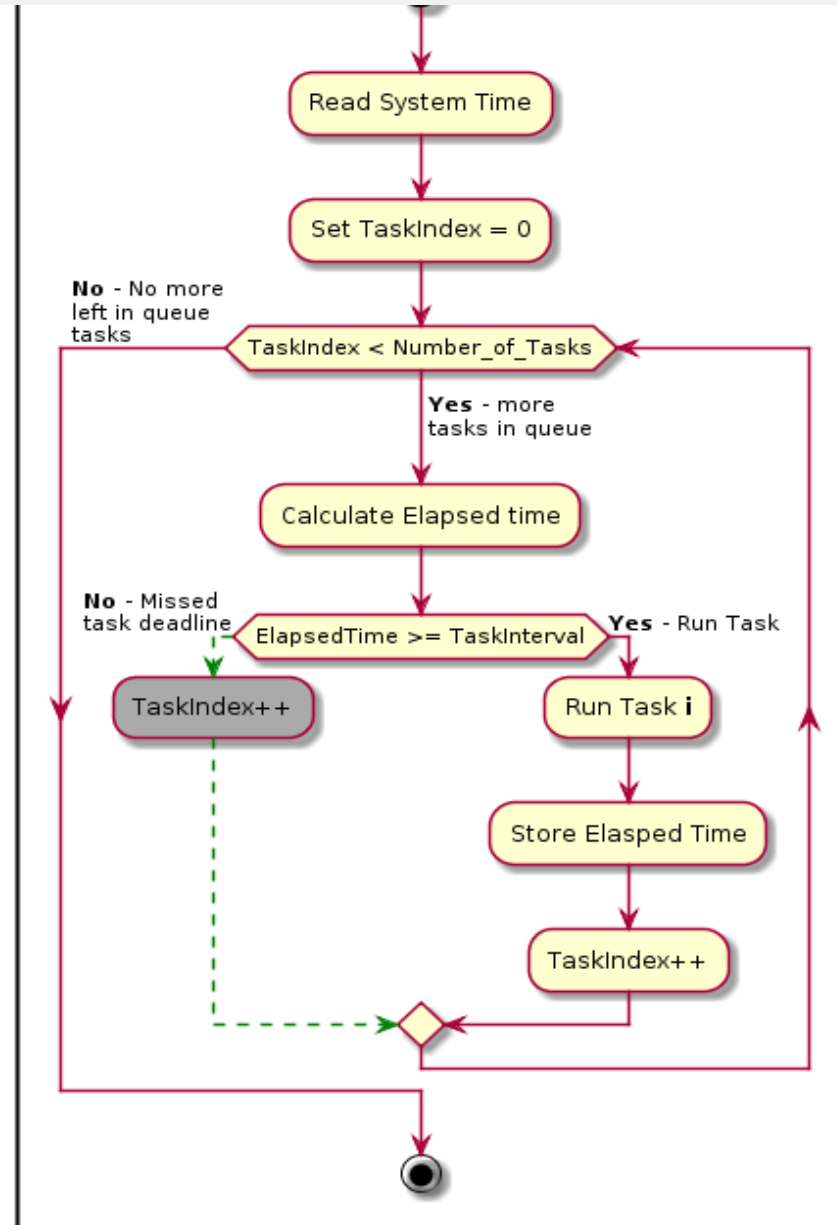
        // Task 2
        Task2(); // takes 250 ms
    }

    printf("All good. Main Terminating.\n");
    return EXIT_SUCCESS;
}
```

- Simple programming structure
 - Execution time is easy to calculate
- What happens when we add more tasks?

Cooperative Scheduling

- Cooperative Scheduling uses an array of tasks and cycles through them with a pointer
 - Completely deterministic
 - Guarantees timing / response
 - However is not a true pre-emptive scheduler as all static tasks have same priority level.
- Compatible with Interrupts
 - Important tasks can be assigned to interrupts
 - Less important tasks are assigned as static tasks in the array loop
 - If time is exceeded less important tasks may be missed
- More complex to implement than round robin/IRQ based scheduling
- Tasks need to be written to execute in shortest time possible to ensure other tasks can get hold of CPU.

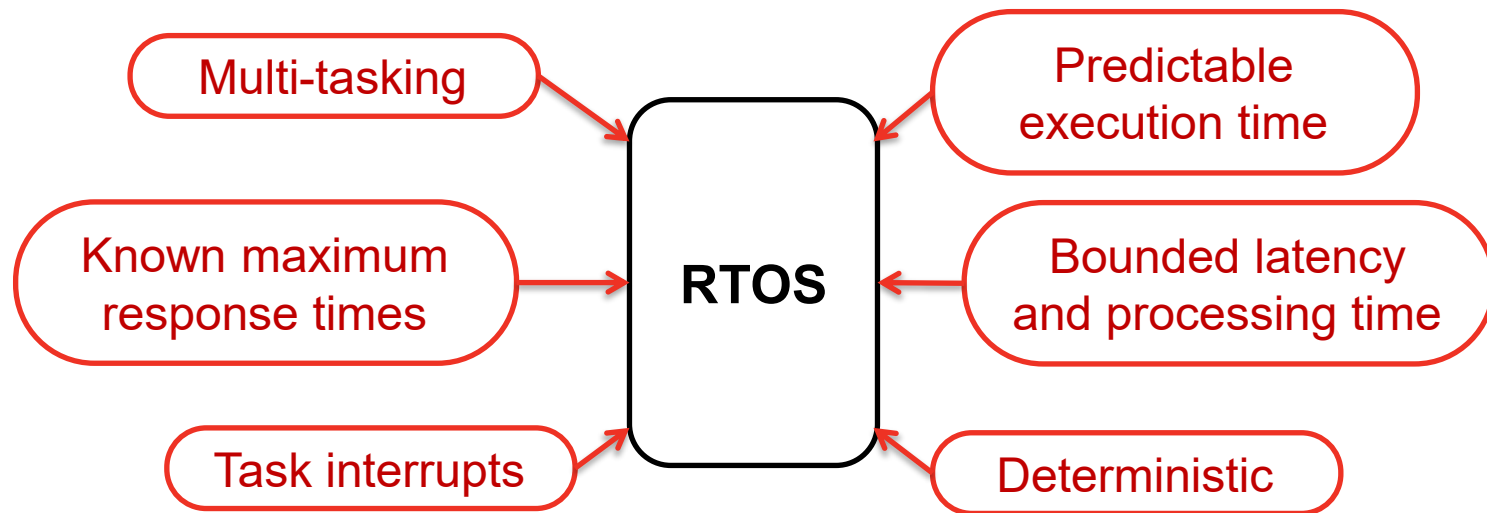


What is a Real-Time Operating System (RTOS) ?

Real-time Operating System (RTOS)

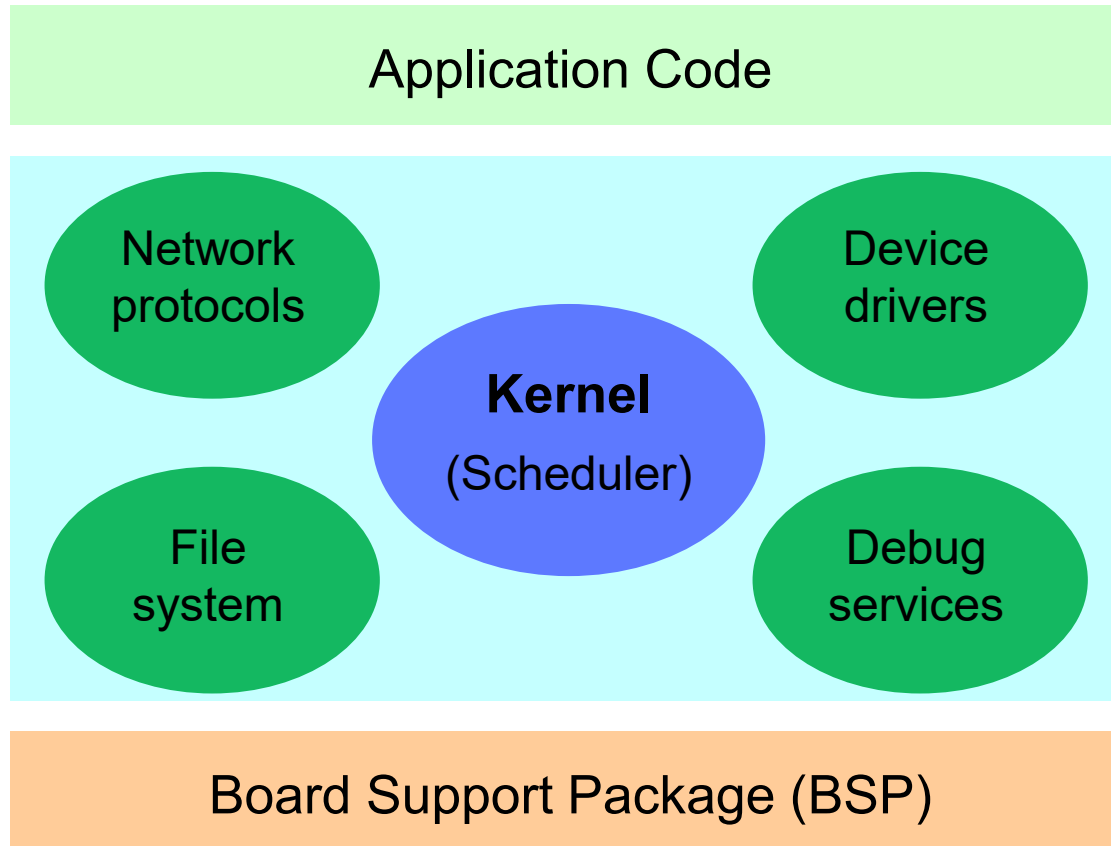
- **Definition:**

A RTOS is an operating system designed to manage hardware resources of an embedded system with very precise timing and a high degree of reliability.



- **Response time of kernel, middleware, tasks & scheduler are well defined**
 - Guaranteed execution/response times
 - Must support preemption – tasks can't be allowed to hog the CPU or other resources
 - Designed for concurrency – task switching support
 - Provide synchronization tools (eg. semaphores, message passing, etc.)

Real-time Operating System (RTOS)



Additional:

- TCP/IP
- Telnet
- FTP
- Shell
- Message passing
- semaphores

Many BSPs:

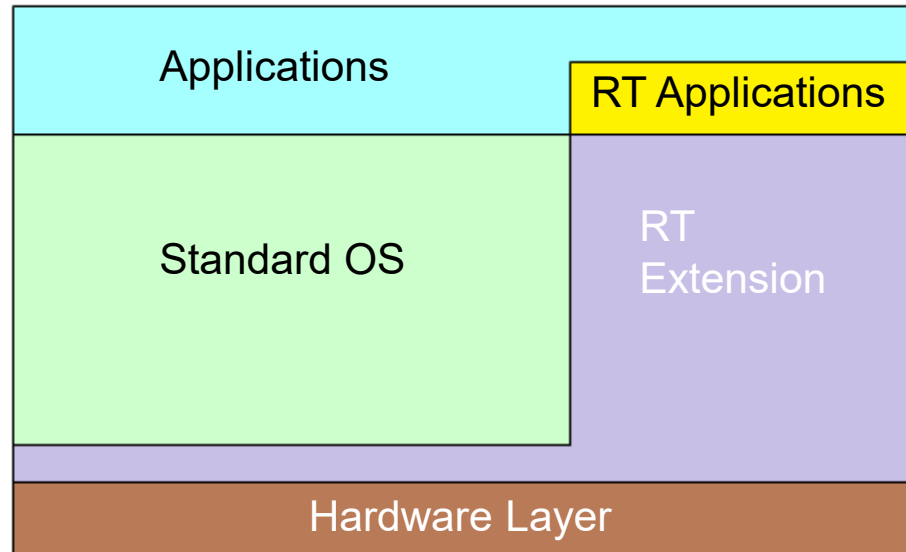
- BeagleBone
- DE10 - Altera
- i.MX6
- X15

- Bare minimum is the Kernel and scheduler. With QNX this can be done.
- Can be added as extensions to conventional operating systems
- Typically conform to a standard specification such as Portable Operating System Interface (POSIX), or at the very least have similar interface functions.

Components of a RTOS

- **Hardware components**
 - Clocks, watch dog timers, etc.
 - Memory management
 - Interrupt service mechanism
 - I/O management, PCI bus, etc.
- **Hardware abstraction layer**
- **Process (task) manager**
 - Scheduler
 - Synchronization mechanism
 - Mutex and/or Semaphores
 - counters
 - Interprocess communications (IPC)
- **Development Environment**
 - Compilers, debuggers, etc.
- **Board Support Packages (BSP)**
 - Support for x86, ARMv7le or other CPU architectures.

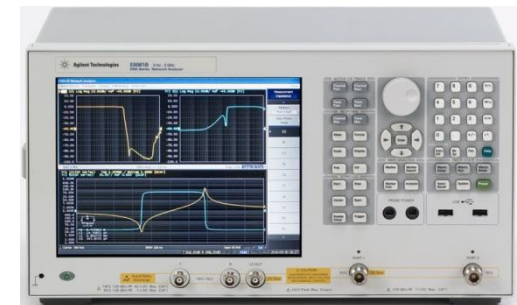
Using RT extensions with an OS



Model for OS such as: RT-Linux, Solaris, Windows Embedded/CE, Windows NT, etc.

Advantages: Rich functionality, works on lots of different hardware, wide support for non-RT part.

Disadvantages: CPU power, memory resources, May not be as fail safe as you need.



[21]

Commercial Real-Time Operating Systems

- **Supports POSIX Standard 1003.1b**

“The ability of the operating system to provide a required level of service in a bounded response time”

- Message queues, threads, timers, etc.
- Set of standards under the ISO/IEEE charter that different OS implement to have common interfaces
- Makes it easier to move/port applications from one RTOS to another

- **Has the following characteristic**

- Consistency, Reliability, Predictability, (known) Performance, Scalability

- **Main Functions:**

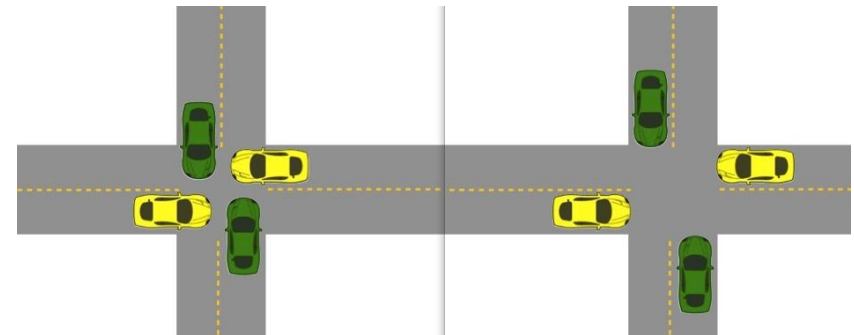
- Resource Allocation, Scheduling, Task management, Interrupt handling

- **Designed to interface with a wide assortment of devices and protocols**

- Interface: Serial/parallel, USB, I²C, PCMCIA, IDE, etc.
- Communications: Serial, ethernet, low bandwidth radio, IrDA, 802.11b devices, etc.
- User Interfaces: LCD, Keyboards, Touch sensors, Sound, Digital pads, Webcams, etc.
- Sensors: fire, temperature, pressure, water level, seismic, sound, vision, chemical, etc.

System Validation

- Must be able to **validate component independently** of the system context
- **Worst case execution times**
 - Establishing **upper bounds on worst performance**
 - This is sometimes impossible to do depending the application, so thought on how the system will cope with deadlock or resource starvation need to be considered:
 - **Deadlock**: Occurs when **two or more competing actions** are each waiting for the other to finish, and thus neither ever does.
 - **Starvation**: Occurs when **a process is perpetually denied necessary resources to process its work**.
 - **Livelock**: A special case of starvation that occurs when **two or more processes continually change their state in response to changes in the other processes**. The result is that none of the processes will complete. (Can happen if all process do the same thing to avoid deadlock, see: <https://en.wikipedia.org/wiki/Deadlock>)
- **Simulations required**
 - generate events which stress the system
 - Test all possible combinations of input
- **Formal verification**
 - Verify individual hardware components
 - verify underlying algorithm



Example of Livelock... no meaningful progress [28]

Synchronization & IPC in QNX – Course Preview

- When concurrent tasks share data you must guarantee **safety** (no corrupted shared data) and correct **mutual exclusion** of critical sections. The course builds up these tools:
- **Protect shared data (one node)**: mutex, reader/writer lock, semaphore (counting), barrier, condition variable – Labs 3 & 4.
- **Pass messages (buffered)**: POSIX message queues (mq_open / mq_send / mq_receive) – work within a node and between nodes over qnet – Lab 2.
- **QNX native message passing (synchronous, client-server)**: MsgSend / MsgReceive / MsgReply via name_attach & name_open, or ChannelCreate & ConnectAttach – lowest latency, real-time guarantees – Lab 6.
- **Watch out for**: race conditions, deadlock, starvation, livelock and priority inversion (Written Assessment & Project).
- *Read more: QNX System Architecture – Synchronization services:
http://www.qnx.com/developers/docs/7.1/#com.qnx.doc.neutrino.sys_arch/topic/kernel_Thread_synchronization.html*

Real-Time Systems Design

- **Real-Time Systems can be designed top-down or bottom-up**

- Top down focuses on the architecture
- Bottom up on the individual components

Uni-processor
Microprocessor
Distributed system

- **Execution Mode**

- **Preemptive** – task may be interrupted by higher priority thread
- **Non-preemptive** – After a task starts executing, no other task can take over.

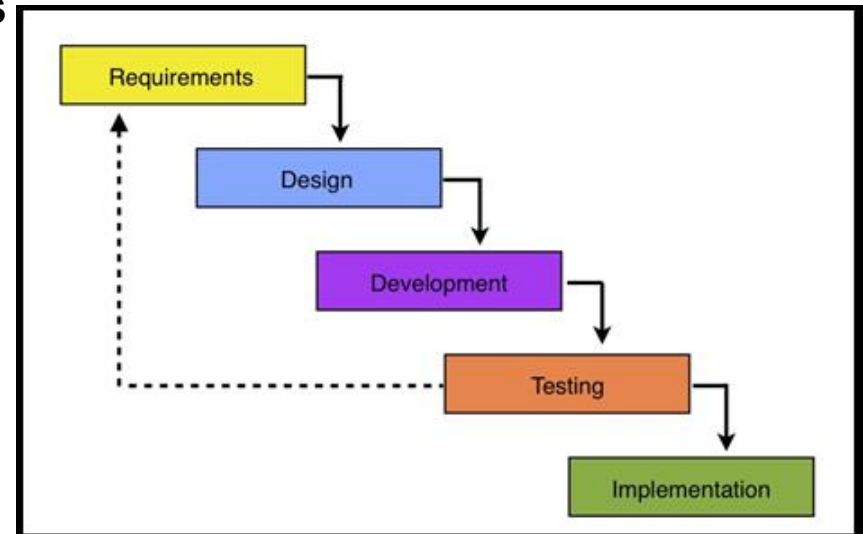
- Follow typical **Software Lifecycle phases**

- System Engineering
- Requirements Analysis
- Detailed Design
- Coding
- Testing
- Maintenance

- **Real-Time Specification & Design**

- State Charts (Finite State Machines)
- UML (concurrent objects)
- Include timing requirements

Subjects of future lectures



[26]

QNX RTOS

What is QNX

- **QNX is a Real-Time Operating System designed for critical tasks**

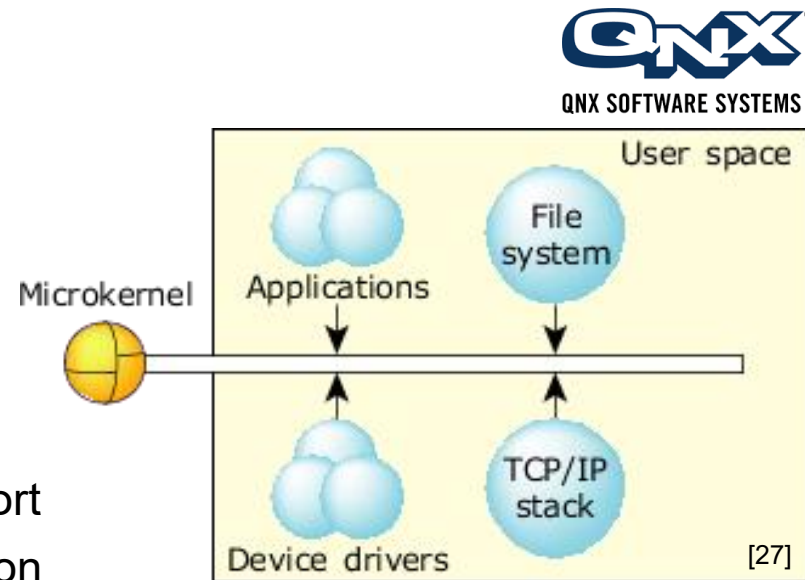
- RTOS Built from the ground up (super small microkernel)
- Aimed directly at the embedded systems market (mission critical)
 - Industrial networks
 - Telecommunications
 - Medical
 - automotive

- Developed by a Canadian company, QNX Software Systems.

- Been around for over +30 years
- Full feature OS, module in design, lots of support
- Was acquired by BlackBerry, Research in Motion

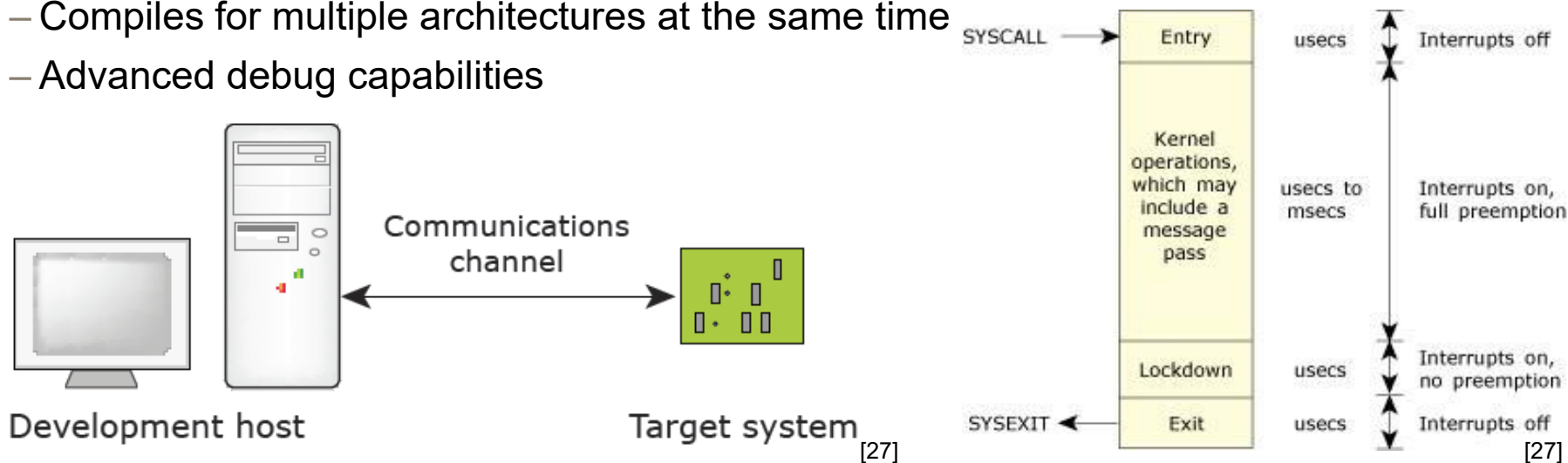
- **QNX is based on the Neutrino microkernel**

- Runs everything as small tasks on a client-server model
- Allows users to turn off any functionality they don't require without changing the kernel
- Very small (ie <400kb) in its smallest size
- Supports multi-core CPUs and is designed for distributed processing.
- **Highly reliable because it is a true microkernel architecture.**



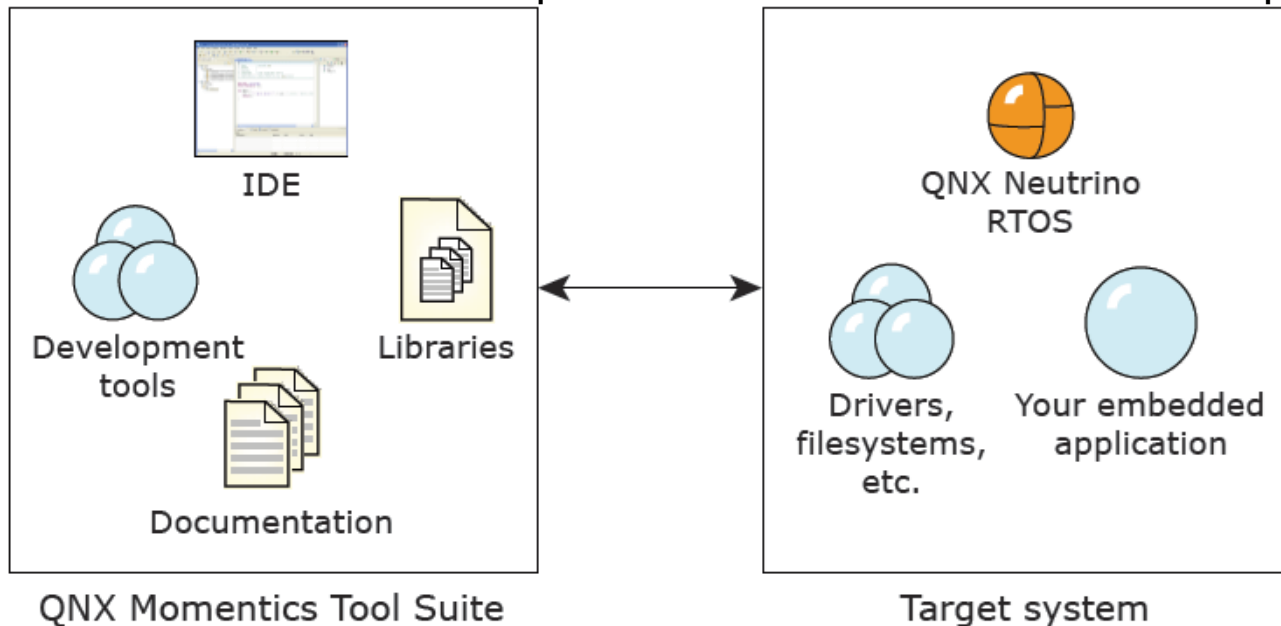
What is QNX

- Under QNX Neutrino, every driver, protocol stack, file system and application runs in the safety of **memory-protected user space, outside the kernel**
 - This is very different to how the Linux, Windows kernels work
- QNX Supports **POSIX-compliant API/function calls**
- Due to its network architecture it is **highly scalable**
 - Need more processing power – add more nodes
- Has **fast context switching** and **low Interrupt latency**
- Has an excellent Integrated Development Environment (IDE) - Momentics
 - File management, transfer and deployment capabilities
 - Compiles for multiple architectures at the same time
 - Advanced debug capabilities



Getting Started with QNX - DEMO

- Create QNX account and Fill in Education End participation google spread sheet
- Obtain PDF files from Canvas
 - **Getting Started with QNX at RMIT** – A Quick Introduction to Virtual Box / VMware Player and QNX Target VMs
- If you need help about QNX:
<https://www.qnx.com/developers/docs/7.1/#com.qnx.doc.ide.userguide/topic/about.html>
- Read getting started guides and start Lab 1 tasks as soon as possible
 - See readme file on Ubuntu Desktop and comment code within the sample project



[27]

Sample Program

Simple example program that displays #CPUs available, Process ID and hostname of target.

```
#include <stdlib.h>
#include <stdio.h>
#include <sys/syspage.h> // for number of CPUs

int main(int argc, char *argv[])
{
    printf("Welcome to the QNX Momentics IDE\n");
    printf("\n"); // print a blank line to the console

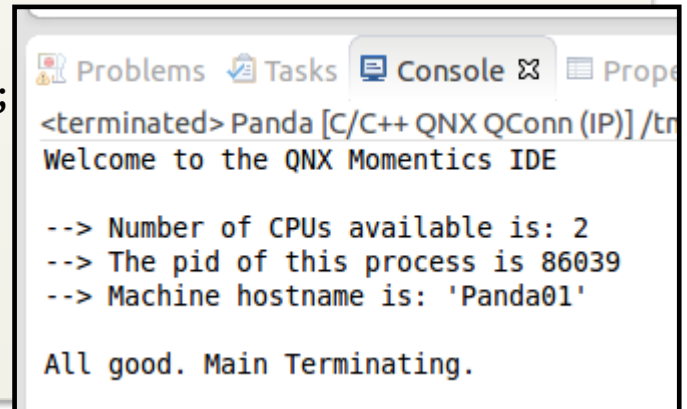
    int num_cpus; // get the number of CPUs
    num_cpus = _syspage_ptr -> num_cpu;
    printf("--> Number of CPUs available is: %d\n", num_cpus);
    printf("--> The pid of this process is %d\n", getpid() );

    char hostnm[100];
    memset(hostnm, '\0', 100);
    hostnm[99] = '\n';
    gethostname(hostnm, sizeof(hostnm));

    printf("--> Machine hostname is: '%s'\n", hostnm);
    printf("\n");

    sleep(10); // wait here for 10 seconds
    printf("All good. Main Terminating.\n");
    return EXIT_SUCCESS;
}
```

Console output (after 10 seconds):

A screenshot of the QNX Momentics IDE console window. The window has a title bar with icons for Problems, Tasks, Console, and Properties. The console text shows the program's execution: it starts with a terminated message, then prints 'Welcome to the QNX Momentics IDE', followed by three lines of output: '--> Number of CPUs available is: 2', '--> The pid of this process is 86039', and '--> Machine hostname is: 'Panda01''. Finally, it prints 'All good. Main Terminating.'.

```
<terminated> Panda [C/C++ QNX QConn (IP)] /tr
Welcome to the QNX Momentics IDE

--> Number of CPUs available is: 2
--> The pid of this process is 86039
--> Machine hostname is: 'Panda01'

All good. Main Terminating.
```

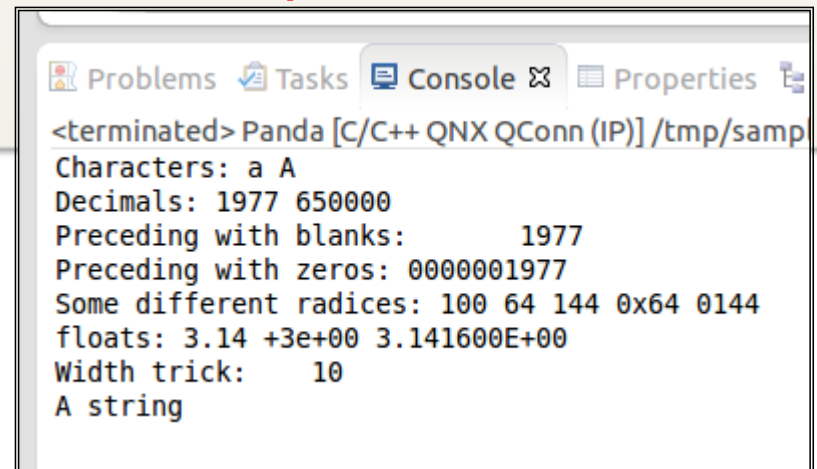
Review C functions

- See: <http://www.cplusplus.com/reference/cstdio/printf/>
- Unlike cout in C++, specifiers are used as arguments within the string to be printed to tell the compiler how to handle the variable/constant being printed to the console.

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("Characters: %c %c \n", 'a', 65);    // %c is for character
    printf("Decimals: %d %ld\n", 1977, 650000L); // %d is for Signed decimal integer
    printf("Preceding with blanks: %10d \n", 1977); // %10d adds 10 leading spaces
    printf("Preceding with zeros: %010d \n", 1977); // %010d fills the spaces with 0
    printf("Some different radices: %d %x %o %#x %#o \n", 100, 100, 100, 100, 100);
    printf("floats: %4.2f %+.0e %E \n", 3.1416, 3.1416, 3.1416);
    printf("Width trick: %*d \n", 5, 10);
    printf("%s \n", "A string");
    return 0;
}
```

Console output :



```
<terminated> Panda [C/C++ QNX QConn (IP)] /tmp/samp
Characters: a A
Decimals: 1977 650000
Preceding with blanks:      1977
Preceding with zeros: 0000001977
Some different radices: 100 64 144 0x64 0144
floats: 3.14 +3e+00 3.141600E+00
Width trick:    10
A string
```

Summary

- RTOS are becoming more viable due to advances in technology.
- The application of the RTOS needs to be considered
 - Soft, Hard, Firm or Weakly Hard
 - Applications such as fuel injection, nuclear reactor control, satellite control, multimedia, data display systems, etc. all have different requirements.
- RTS require different design and verification techniques to non-RT systems
 - Probably not dedicated to a single task – most likely using distributed processing
- Executes in precisely the same amount of time every time
 - Handles multiple competing events
 - Events can occur simultaneously
- User has control of OS policies
- Highly responsive
 - Fast task switching
 - Fast interrupt response
- Limited security/protection



[29]

Appendix

Supplementary reference material – extra background and deeper detail for self-study.

Additional Features of a True RTOS

- **High Resolution Clock and Timer Support**
 - Highly important
 - Hard real time applications require timers services with resolution in the microseconds
- **Predictable and Fast Interrupt Latency**
 - Interrupt latency is the time delay between the occurrence of an interrupt and the running of the corresponding Interrupt Service Routine (ISR).
 - Upper bound on interrupt latency must be bounded (ie. Guaranteed performance)
- **Real-Time Priority Levels**
 - Must support static (or real-time) priority levels
 - Traditional OS dynamically change priority levels to maximize system throughput
- **Fast Task Preemption**
 - When a high priority task arrives, lower priority tasks should preempt
 - The lower the “task preemption time”, the better.
- **Support for Resource Sharing (including Asynchronous I/O)**
- **Memory Management**
 - Large systems might need Virtual memory support, others don't need it.

Software Based RT Systems and Safety Critical Systems

- In safety critical systems failure to respond in time is not an option.
 - Hard real time design approach was the only option (High cost)
 - This could involve some form of backup system which can take over when the software system failed – **multiple redundancy subsystems**
- Current trends are to have wholly software controlled RT systems. Partially due to:
 - Availability cheap and readily available microprocessors
 - Complexity of systems (they are now required to do a lot more)
 - User now demand advanced graphic user interfaces (GUI).
 - Network communications requirements
- Control peripheral devices and interfaces
 - Interrupt service handlers
 - DMA (Direct Memory Access) controls
- Usually multi-tasking
 - Multiple processes/tasks
- React within real-time deadlines
 - Give priority to critical tasks
 - Use Real-Time Scheduling algorithm



Real-Time Systems Design

- **Real-time systems are typically single-purpose**

- Lots of process/threads with a common aim
- Could be many nodes functioning together with a common aim

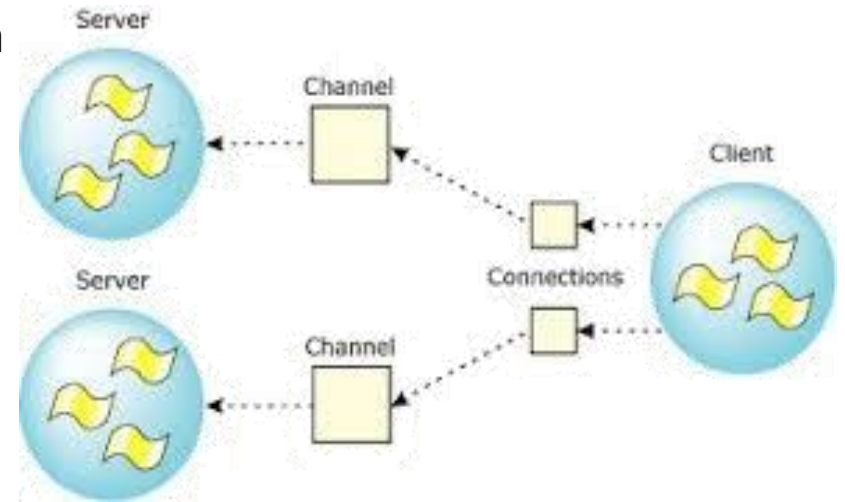
- Often, RT systems **do not** require any **user input interface**... just a display

- The hardware requirements for a real-time system are often a **lot less substantial** than a typical desktop PC... not always the case but they do typically have a smaller footprint

- There are higher overhead requirements for **protected memory** due to the increased context switching and overall application structure used in RT systems designs

- Any use of **memory paging can increase context switching times** – this can delay the time required to meet a deadline – Task switching is more efficient from this perspective.

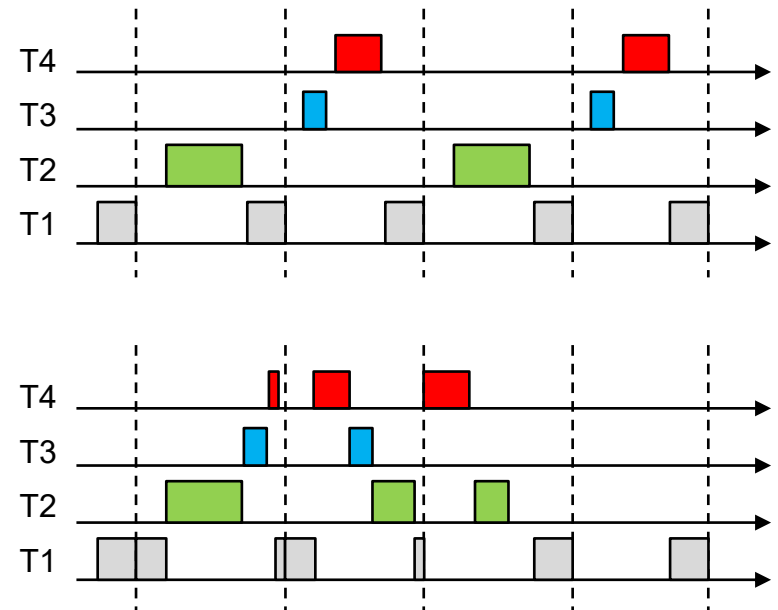
- The design/architecture of software applications in a RT system is **far more fragmented** than desktop applications. Again this is due the different design concepts employed when developing a distributed RT application.



Server-client model used in RT applications [27]

Real-Time Scheduling and Task Switching

- **Scheduling task is the primary means that an OS meets task deadlines**
- Lots of work has been done in the area of developing appropriate schedulers for meeting real-time deadlines.
 - Scheduling for uniprocessors (single core)
 - Scheduling for multi-core preprocessors
 - Scheduling for distributed systems (over networks, etc.)
- **Fast context switching also play a vital key**
- **Scheduling decision techniques include:**
 - **Clock Driven**
 - State table
 - Cyclic (e.g. feedback control loops)
 - **Even Driven**
 - Priority based
 - Rate Monotonic Analysis (RMA)
 - Earliest Deadline First (EDF)
 - **Hybrid**
 - Round-robin (time slicing)
 - Sporadic (complex budget timing)



List of Common RTOS

Dedicated RTOS

- QNX Software Systems – QNX Neutrino
- Wind River Systems – VxWorks
- Mentor Graphics - VRTX
- Palm Computing – PalmOS
- Mobile OS – SymbianOS
- LynxOS – for hard real-time systems (28KB)
- ThreadX
- RTEMS www.rtems.org

Linux Based RTOS or extensions

- Embedded Debian
- ETLinux
- uCLinux (for non MMU processors)
- FreeRTOS
- eCos
- RT Linux
- OSE
- SUSE RT Extension

Window Based RTOS

- WindowsCE, NT, etc. (can be very small 400KB)
- IntervalZero's RTX64 and RTX turns Windows into an RTOS

Java base RTOS

- Jbed RTOS package

Object-oriented RTOS

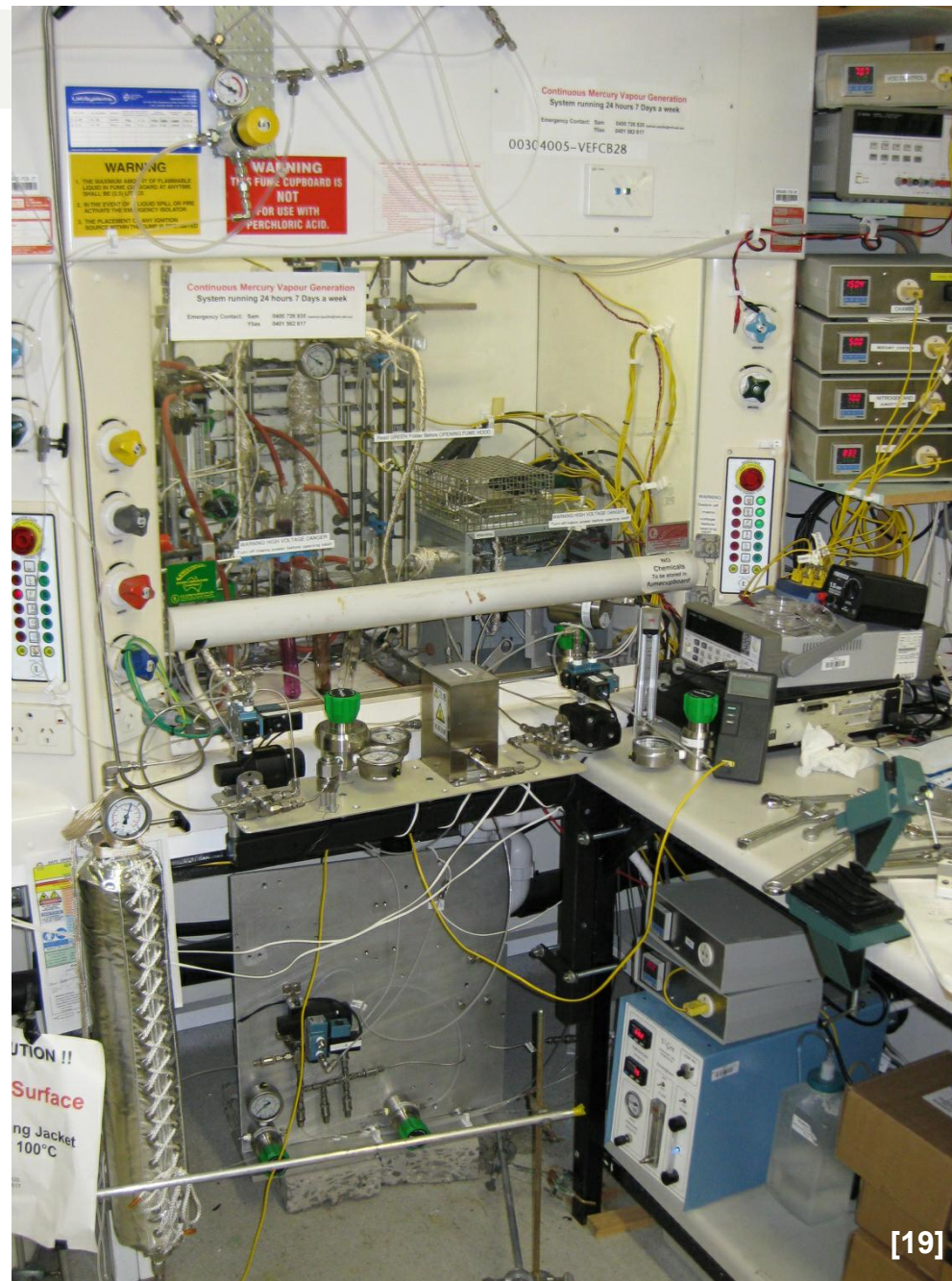
- Wind River Systems – pSOS

Features that may be missing from a RTOS

- Support for specific peripheral devices
- Protection and security mechanisms
- Multiple users – some have support
- Multiple modes
- Advanced memory allocation
 - Dynamic allocation

RMIT's Mercury Systems

- Mission critical system to deliver mercury vapour to test the performance of novel chemical sensors.
- System has:
 - 9 Digital mass flow controllers
 - 2 High temp pressures sensors
 - Controllable humidity generator
 - 6 electro-pneumatic valves
 - 12 PID control modules 240V heat trace tapes and reactor coils.
 - 4 sensor data acquisition ports
 - 8 temperature sensors
 - 8 relay ports, etc.
- Failure could cause fire, poisoning (illness or death), loss of data, loss of refinery sample gas (\$\$\$), etc...



[19]

What is an Operating System?

- **Similar to a government**
 - Performs no useful function by itself
 - Provides an environment within which other programs can do useful work
- **Operating system is a resource allocator**
 - Resources (hardware and software) include
 - CPU time
 - Memory space
 - File storage space
 - I/O devices
 - Allocates and manages resources so that the computer system operates fairly and efficiently
 - and meets deadlines if a real-time system
- **System Control**
 - Operating System also acts as a control program
 - To control the various I/O devices and user programs
 - Traps errors and prevents improper use of the computer



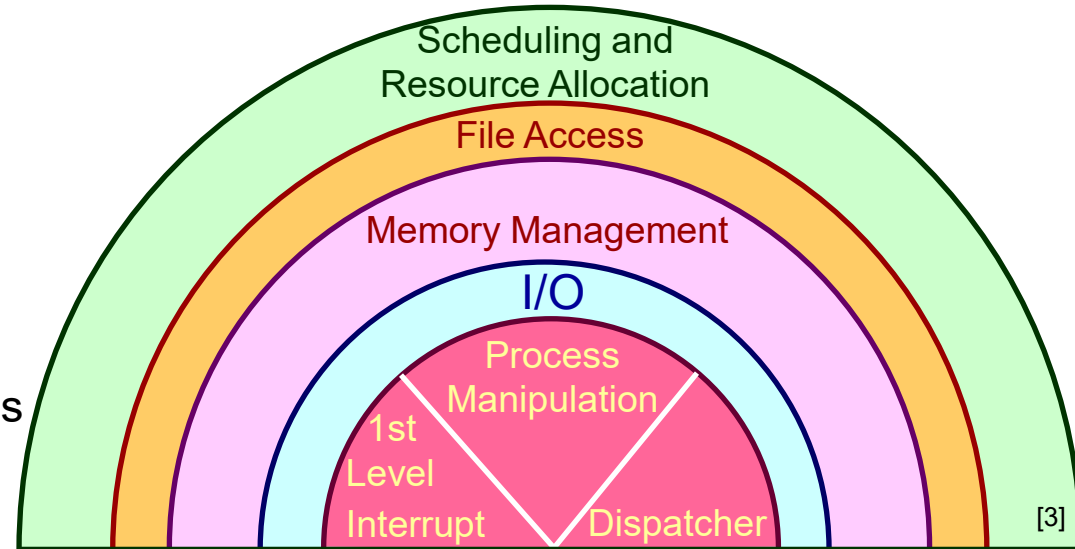
[22]



An error message is far better than a system crash/reboot

Operating System Kernel and OS Layers

- **The Dispatcher**
 - Low level scheduler
 - Performs actual process switch
 - **Process Manipulation**
 - Intermediate level scheduler
 - Moves processes between queues
 - **First Level Interrupt Handler**
 - Initial context switch
-
- **I/O system**
 - Actual device support
 - Reading and writing disk blocks
 - Terminal I/O
 - Printer output
 - Initial handling of I/O errors

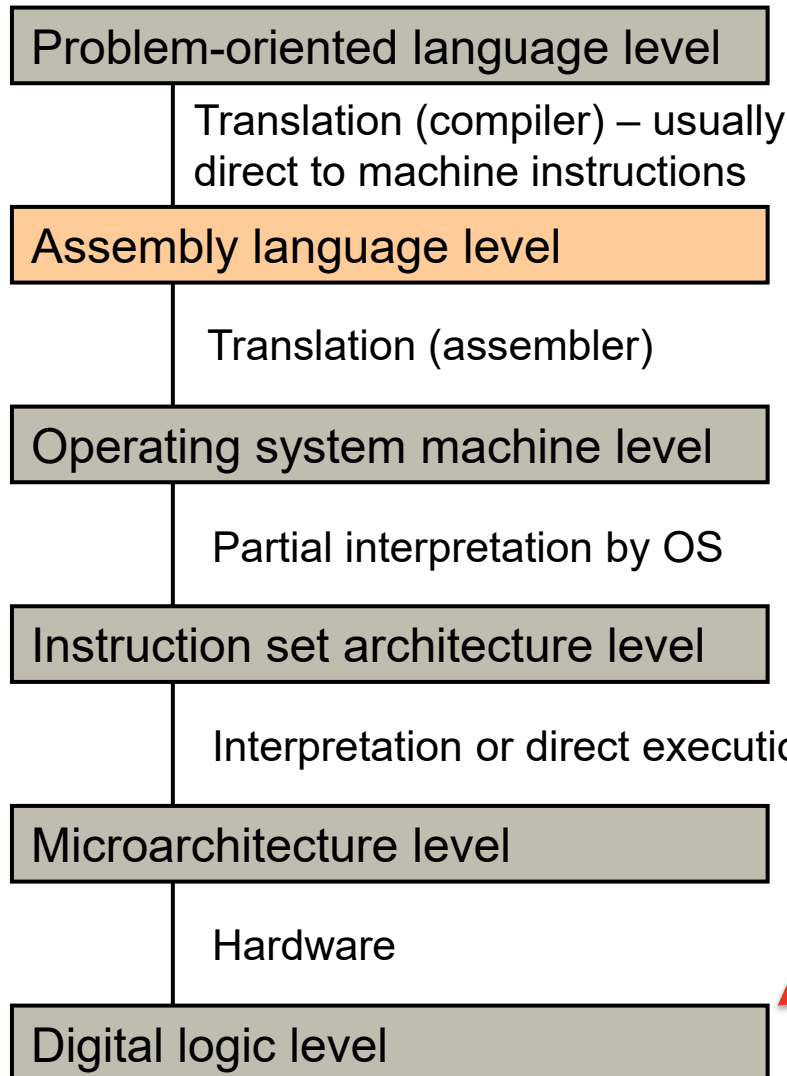


“Layered Model” of Operating Systems

- **Memory management system**
 - Handles memory allocation
 - Segment placement
 - Page or segment replacement
 - Swapping (pages or segments) memory to disk

Operating System from a Programmer's Point of View

Six level model



Where we normally see our selves operating as application based programmers

Where we are going to have to start operating here if we want to start controlling exactly how our programs executes

Need to be here too if we want to interface to the real world

Source: “*Structured Computer Organization*” by A.S. Tanenbaum

What a Real-Time Computing isn't...

Before moving on, it is important to define what Real-Time Computing isn't.

- **Real time computing is not equivalent to Fast computing**
 - Its not the realm of supercomputers
 - Its to be used in low power embedded devices (ie, mobile phones, etc.)
- **Real time systems should not be used for static problems**
 - They are suited to solving dynamic problems
 - Used to control complex environments
 - Overkill for static environments – maybe you need the reliability
- **Real time systems are not used to write device drivers, priority interrupt handlers or the realm of assembly code or low level C**
 - Sure you can do this, but they are designed for high level system development
 - Not low level drivers or controllers.

When should you use an RTOS

- When its easier to write and application in RTOS because the use of the kernel enforces a certain discipline (or structure) in how your code is structured.
- When concurrency is occurring – more than one thing at a time. Using a normal OS will just lead to more complex software as you can't use inbuilt kernel features.

References

Images and content used in this presentation can be found:

- [1] <http://apod.nasa.gov/apod/ap060522.html>
- [2] <http://www.cnet.com/au/pictures/inside-a-power-grid-control-room-photos/>
- [3] Modern Operating Systems, Tanenbaum, Andrew S., Prentice-Hall
- [4] <http://www.smh.com.au/business/markets/stocks-tipped-to-open-higher-as-market-prices-in-reserve-bank-rate-cut-20150503-1mynpl.html>
- [5] Alcoa Alumina – RMIT Student Trip 2010
- [6] <http://www.engineersgarage.com/articles/embedded-systems>
- [7] <http://www.cse.unsw.edu.au/~cs9242/08/lectures/09-realtimex2.pdf>
- [8] http://www.electronicproducts.com/Power_Products/Power_Management/Efficient_dimming_for_LED_lighting.aspx
- [9] <http://nixtalk.net/webserver-rk3188>
- [10] <https://www.engineersaustralia.org.au/portal/news/lab-chip-delivering-laboratory-technology-real-world>
- [11] <http://www.westerndailynews.com.au/Fighter-pilots-tomorrow-use-iPad-style-controls/story-21443924-detail/story.html>
- [12] <http://www.logicsupply.com/explore/io-hub/beaglebone/>
- [13] <http://www.circuitstoday.com/interfacing-hex-keypad-to-arduino>
- [14] <http://www.meas-spec.com/product/pressure/85BSD.aspx>
- [15] <http://www.hiseamarine.com/industrial-valves/>
- [16] <http://www.amazon.com/AGPtek%C2%AE-Universal-Temperature-Controller-Fahrenheit/dp/B007MMOEWY>
- [17] <http://www.buildcircuit.com/ping-ultrasonic-range-finder-using-arduino-board/>
- [18] <http://www.imagesco.com/servo/smc05.html>
- [19] RMIT, CAMIC, Mercury Removal and Industrial Chemical Sensor Group
- [20] <http://www.electronicweekly.com/news/micrium-brings-no-cost-rtos-to-maker-community-2016-02/>
- [21] <http://www.digchip.com/datasheets/3270045-e5061b-e5061b-network-analyzer.html>
- [22] <http://www.babeltechreviews.com/33-game-platform-analysis-devils-canyon-gtx-980-sli/>
- [23] http://www.teacherclick.com/winxp/t_3_1.htm
- [24] https://en.wikipedia.org/wiki/Wireless_router
- [25] <http://www.freertos.org/implementation/a00004.html>
- [26] <https://cio.gov/software-development-life-cycle/>
- [27] QNX 660 Documentation
- [28] <http://15418.courses.cs.cmu.edu/spring2013/article/27>
- [29] <http://www.engadget.com/2013/01/08/qnx-concept-bentley-continental-gt/>

Acknowledgment

- This course builds on the original RMIT Real-Time Systems notes by **Roy Ferguson** and **Dr Samuel Ippolito** (RMIT Melbourne).
- **QNX diagrams and documentation** courtesy of QNX Software Systems and BlackBerry.
- *Questions are always welcome. Enjoy building real-time systems.*